The SCSI/Link Implementor's Guide

DaynaPORT SCSI/Link (SL003) wire protocol and the behavior of all known implementations

Ingo Paschke

August 2026 – version 0.96

Abstract

The DaynaPORT SCSI/Link is a SCSI-attached Ethernet adapter for machines whose only fast expansion bus is SCSI. The protocol was never publicly documented. This guide describes the complete command set as implemented by the SL003 v2.0 ROM and, per command, the compliance of every known implementation: the v1.3 ROM, BlueSCSI v2, the BlueSCSI-SL003 fork, ZuluSCSI, PiSCSI, and the Snow emulator. The v2.0 ROM is the reference; "compliant" means indistinguishable from it on the wire for that command.

Contents

1 Introduction

1.1 Scope

Implementations covered, with the source each entry is based on:

- **SL003 ROM v2.0** – from the disassembly; the reference for this guide.
- **SL003 ROM v1.3** – differences known from the v2.0 disassembly's change markers.
- **BlueSCSI v2** – upstream `network.c`.
- **BlueSCSI-SL003** – firmware fork reimplementing the v2.0 ROM behavior, one documented extension.
- **ZuluSCSI** – `network.c`, shared SCSI2SD lineage with BlueSCSI; wire behavior identical to it.
- **PiSCSI** (formerly RaSCSI) – `scsi_daynaport.cpp`, Linux TAP based.
- **Snow** – Macintosh emulator, `ethernet.rs`.

Byte offsets are zero-based. Multi-byte wire fields are big-endian unless stated otherwise (the statistics counters are the exception). "Record" means one length-prefixed unit in a READ response (section 2.3); "frame" means the Ethernet frame inside it.

1.2 Device model

The adapter is a SCSI target (any ID; commonly 0) with a single LUN and INQUIRY device type 3 (processor). It queues received Ethernet frames internally and transmits frames handed to it via WRITE. There is no interrupt line; hosts poll. Two pieces of state gate the command set:

Enabled Set by `0x0E`. While disabled, only TEST UNIT READY, REQUEST SENSE, INQUIRY and ENABLE are accepted; everything else answers CHECK CONDITION. Enabling resets the Ethernet controller and discards queued receive frames.

Mode set Set by the mode flavor of `0x0C`. Its only observable effect is that the runt flag (`0x80`) appears in record headers only once a mode has been set.

2 SCSI commands

All commands use 6-byte CDBs, drawn at the head of each section. Allocation lengths are big-endian. Bytes shown as zero must be sent as zero; WRITE rejects nonzero values there.

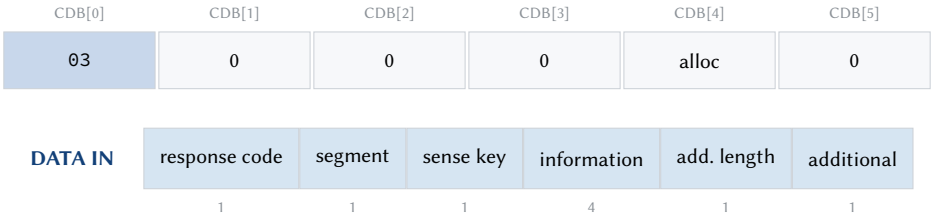
2.1 `0x00` TEST UNIT READY

CDB[0]	CDB[1]	CDB[2]	CDB[3]	CDB[4]	CDB[5]
00	0	0	0	0	0

No data phase.

GOOD, also while disabled. Every implementation complies. It therefore cannot serve as a readiness probe after ENABLE (section 3.1).

2.2 `0x03` REQUEST SENSE



The answer is standard fixed-format sense data, cut to nine bytes. On the ROM (handler `0x0b8c`) an allocation of zero defaults to four and anything larger is clamped to nine; the device never answers more.

Byte	Field	Contents
0	response code	`0x70`, current error, fixed format. The ROM never sets the valid bit `0x80`, so the information field never means anything
1	segment number	zero
2	sense key	low nibble; the only field the ROM fills
3–6	information	zero on the ROM
7	additional sense length	zero on the ROM; emulators serving their framework's standard sense report the standard trailing bytes here
8	additional sense bytes	first byte of; zero on the ROM

The sense key in byte 2 is the only portable field. Key 5 (illegal request) is the answer to malformed CDBs, to refused command flavors, and to data commands while the interface is disabled. The SL003 fork answers key `0xB` (aborted command) after a SCSI parity error. The emulators serve full 18-byte fixed-format sense truncated to the allocation, so a nine-byte request reads the same layout there, with ASC and ASCQ out of reach beyond byte 8.

Drivers should issue REQUEST SENSE after any CHECK CONDITION; a sense indicating a freshly reset interface calls for a re-run of the bring-up sequence (section 3.1).

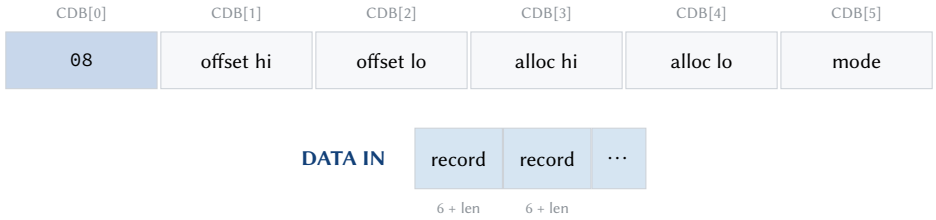
Implementation	Status	Notes
ROM v2.0	✓	reference
ROM v1.3	✓	
BlueSCSI v2	◗	generic SCSI2SD sense format, standard length
BlueSCSI-SL003	◗	same; served before the personality runs
ZuluSCSI	◗	same as BlueSCSI
PiSCSI	◗	generic framework sense
Snow	◗	generic

✓ matches the v2.0 ROM ◗ differs, usable ✗ not implemented

Table 1: REQUEST SENSE compatibility

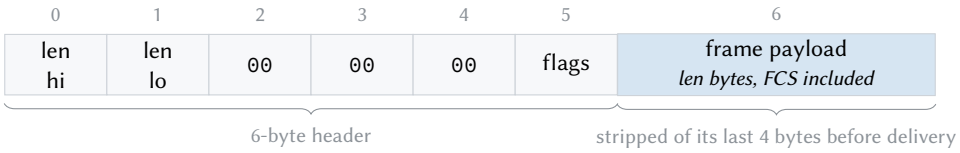
The deviations are harmless: every implementation reports a sense key in byte 2, which is all a driver should rely on.

2.3 `0x08` READ



The record format

Every READ response is a sequence of records:



- `len` counts the frame *including its 4-byte FCS* (the Ethernet frame check sequence, the CRC32 that trails every frame on the wire); the maximum is therefore 1518. Clients of a driver expect frames without the FCS; strip the last 4 bytes before delivery.
- `len = 0`: the receive queue is empty, or (blind mode) the record terminates a batch the device cut short.
- `flags` bit `0x10`: more packets are queued in the adapter. The v2.0 ROM sets it truthfully in both receive modes.
- `flags` bit `0x80`: runt frame; reported only once a mode has been set.
- The four bytes after the length all `0xFF`: observed on real hardware after the adapter dropped a packet (its internal buffer is about 6 KB); the length field then reads a nonsense value and the condition persists until a disable/enable cycle. Emulators do not reproduce this. Treat it as a signal to re-run enable.

A driver's reaction to `len = 0` is the same whether it marks an empty queue or a cut-short batch: end the transaction and poll again later; the two cases need not be distinguished.

What the host's SCSI interface must do

Polled and bounded mode need nothing unusual: one command, one data-in of at most N bytes, and a device that may send fewer. The host asks for a fixed size, the device sends what it has and ends the data phase, and the records are parsed out of the buffer afterwards. Every common API does this – SCSI Manager 4.3, Atari SCSIDRV, Linux `SG_IO`.

Blind mode is the exception. Nothing bounds its batch in advance, so the host cannot name a transfer size that is both safe and sufficient; it has to read each record's header, learn the length, and read the payload, all inside one open command. That requires a SCSI engine able to issue *several data-in transfers per command*. Where the engine cannot, blind mode is unavailable and the choice is polled or bounded.

Length validation

The record length field is untrusted wire data. Validate it against 1518 before transferring the payload. On nonsense, transfer nothing further, wait for the target to leave the data phase on its own, and then complete the transaction (section 3.5).

Timing

The real device pauses about 75 μs between a record header and its payload and about 300 μs between records (bus analyzer measurement; the ROM contains no programmed delays – this is Z180/DMA turnaround). Software-timed blind transfers on slow hosts depend on these gaps.

Receive modes

The mode byte selects how much arrives:

Value	Mode	Contract
`0x80`	polled	exactly one record per command
`0xC0`	blind	records until the device ends the batch
`0xE0`	bounded	blind, batch capped by the allocation length (extension)

Within the mode byte, bit `0x80` is always set, `0x40` selects blind, and `0x20` is the bounded extension – ignored by real ROMs, which is why it must be probed (section 2.9). On the ROM the allocation length clips *each record*, not the batch; use at least 1524.

Bit `0x10` (BlueSCSI-SL003 only) requests *seamless emission* – the whole answer in one burst, no pacing gaps; see the extension's own section below (section 2.3).

The offset field is a stateless peek/resume into the head packet, in polled mode only: a driver may re-read a record it has already partly taken. It is ROM behavior (v2.0 disassembly, `0x0e05–0x0f17`); of the emulators only the BlueSCSI-SL003 fork implements it. Ordinary drivers send zero.

Figure 1 contrasts what one poll costs in each mode when four frames are waiting.

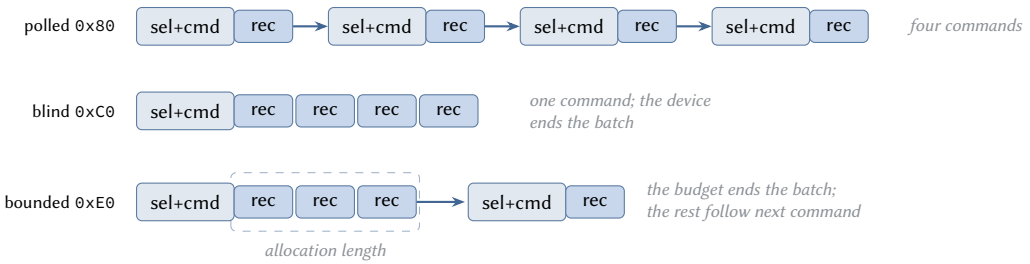


Figure 1: Four frames queued. Polled mode pays a fresh arbitration and selection per record. Blind mode takes all four in one command, the device deciding where the batch ends. Bounded mode takes as many as the allocation length allows and leaves the rest for the next command.

Polled mode

One record per command, regardless of the more-flag. The flag says another *command* should follow, not another record in this transfer. One command, one transfer, header parsed from the front of the buffer.

Implementation	Status	Notes
ROM v2.0	✓	reference: exactly one record
ROM v1.3	✓	
BlueSCSI v2	✓	one record; 75/300 μs gaps present
BlueSCSI-SL003	✓	gaps configurable; honors `CDB[1..2]` as a peek offset
ZuluSCSI	✓	identical to BlueSCSI v2
PiSCSI	◗	one record, as required, but the reported length is padded to a minimum of 128, and the allocation length is parsed from the low byte alone
Snow	◗	mode byte ignored: answers with up to 8 records even in polled mode; a driver that reads exactly one record per command still works, the rest wait

✓ matches the v2.0 ROM ◗ differs, usable ✗ not implemented

Table 2: READ, polled mode (`0x80`) compatibility

Blind mode

Records back to back inside one data phase. The host must not end the batch; the device ends it, either with a record whose more-flag is clear or, at its record cap, by appending a zero-length terminator record. The v2.0 cap is 200 records; v1.3 has none. Ending the transaction while the target is mid DATA IN can wedge the SCSI bus. If the data phase ends after a complete record even though the more-flag was set, treat it as end of batch rather than an error; PiSCSI behaves this way.

Blind mode is the only mode with a transport requirement beyond a plain SCSI API (section 2.3): the batch length is unknown in advance, so the records must be read one at a time inside one open command. A 6-byte header buffer plus a single 1518-byte payload buffer, reused per record with each frame consumed before the next read, serve batches of any size; the consumer needs a refusal path (drop the frame), because a record must be read off the bus either way.

Implementation	Status	Notes
ROM v2.0	✓	reference: streams to a 200-record cap, then appends a zero-length terminator
ROM v1.3	◗	no record cap at all; batches are unbounded
BlueSCSI v2	◗	stops at a byte budget of two maximum-size records, so several small frames may fit; the allocation length acts as a batch budget; the more-flag means "another record follows in this transfer" and is suppressed when stopping; no terminator record
BlueSCSI-SL003	✓	cap and terminator as the ROM
ZuluSCSI	◗	identical to BlueSCSI v2
PiSCSI	◗	one record per command regardless of mode: the data phase ends after it even with the more-flag set
Snow	◗	up to 8 records; more-flag means continuation, no terminator record

✓ matches the v2.0 ROM ◗ differs, usable ✗ not implemented

Table 3: READ, blind mode (`0xC0`) compatibility

Bounded mode (extension)

BlueSCSI-SL003 only. Bounded mode is blind mode for hosts that cannot do the inline reads: a SCSI engine that binds command, one data-in and status into a single call can never walk a batch record by record on the bus, and blind mode's unknown batch length locks such hosts out (section 2.3). Everything about `0xE0` follows from removing that one obstacle. The host cannot stop the batch mid-phase, so the device must: the allocation length becomes a byte budget the batch never exceeds, the whole batch arrives in one transfer, and the records are walked in memory afterwards.

The batch itself ends exactly the way a blind batch ends. A drained queue: the last record's more-flag is clear and nothing follows. The byte budget, like the record cap: records keep their truthful flags and the zero-length terminator closes the batch – the ROM's own record-cap idiom, with one addition: the terminator's byte 2 reports the queue depth (records, clamped to 255), so a driver can pace adaptively – chase a deep backlog immediately, let a shallow one grow into a worthwhile batch. Because these are the blind-mode semantics unchanged, nothing special is needed for safety, and hosts that ignore the depth byte see ROM behavior exactly.

Probe INQUIRY byte 36 bit `0x01` before use; without the bit, `0xE0` is plain blind. A host that pairs bounded with seamless emission checks bit `0x02` as well (section 2.3). Because the device respects the budget, a batch arrives in one command and one transfer of the buffer size, with the records parsed out afterwards – no special SCSI engine needed.

Implementation	Status	Notes
ROM v2.0	✗	bit `0x20` ignored; behaves as blind
ROM v1.3	✗	as above
BlueSCSI v2	✗	as above
BlueSCSI-SL003	✓	the extension: allocation length bounds the whole batch, so a batch fits one command and one transfer; the closing terminator's flags byte reports the queue state
ZuluSCSI	✗	as above
PiSCSI	✗	as above
Snow	✗	as above

✓ matches the v2.0 ROM ◗ differs, usable ✗ not implemented

Table 4: READ, bounded mode (`0xE0`) compatibility

Crosses here mark an absent extension rather than a defect. The INQUIRY capability bit is what selects this mode, and the probe keeps a driver from sending `0xE0` to the other rows.

Seamless emission (extension)

BlueSCSI-SL003 only, any read mode: bit `0x10` in the mode byte asks the device to emit the whole answer as one seamless transfer – in a blind or bounded batch that is every record of the batch in a single burst, not one burst per record. Without the bit the device keeps the ROM's classic timing: header, 75 μs, payload, 300 μs, next record.

The distinction matters. A pause between two records is a stall exactly as a pause inside one is, so emitting record by record leaves a stall at every record boundary and a host that declared none is still exposed. The effect hides on bulk transfers, whose batches carry a few full-size records, and appears on acknowledgement traffic, where 64-byte records put a boundary every seventy bytes.

The bit exists because the two host camps need opposite wire behavior. Software-timed hosts (Macintosh Plus, SE/30) calibrated their blind loops against the ROM's pauses and depend on them. Hardware-handshaked hosts neither need pacing nor, on Quadra-era SCSI Manager 4.3 machines, tolerate

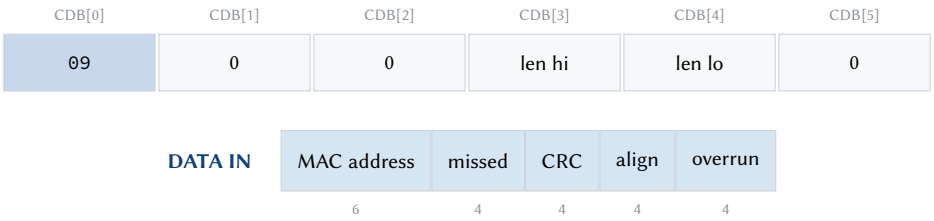
the pauses: an undeclared mid-phase stall makes that host fabricate two zero bytes into its buffer (section 3.4). A host that handshakes in hardware should send the bit; a software-timed host must not; real ROMs and the other emulators ignore it, and the classic timing they provide is what such hosts want anyway.

The capability is advertised in INQUIRY byte 36 bit `0x02` (section 2.9); gate the CDB bit on it, and never on the bounded bit – a device honoring the bound while ignoring the seamless request would keep its pauses, which is exactly the case the probe exists to catch.

The device pays for it with a staging buffer the size of the allocation it is willing to serve, since the batch is assembled before any of it goes out and is never split. An unbounded blind batch that would outgrow the staging area ends early instead, the way the ROM ends one at its record cap; the remaining records lead the next command.

A driver written against the contracts above works on every row. One that assumes the allocation length bounds a blind batch works only on BlueSCSI and ZuluSCSI; one that infers polled behavior from the flag alone breaks on Snow; one that reads blind batches to the terminator without handling an early phase end breaks on PiSCSI.

2.4 `0x09` READ STATISTICS



The counters are *little-endian* 32-bit: the Z180 stored them that way and the ROM hands them out untouched.

Offset	Size	Field
0	6	MAC address
6	4	frames missed
10	4	CRC errors
14	4	frame alignment errors
18	4	overruns (22-byte answers only)

The v2.0 ROM answers 22 bytes. Request 22 and fall back to 18 on a short or failed transfer; the MAC occupies the first six bytes either way. This is also the only reliable way to learn the adapter's MAC.

The counters are *read-and-clear*: every `0x09` zeroes them, regardless of how much the host takes. Read the MAC once at bring-up; a driver that polls `0x09` casually destroys its own statistics deltas.

Implementation	Status	Notes
ROM v2.0	✓	reference
ROM v1.3	✓	
BlueSCSI v2	◗	18 bytes; counters always zero
BlueSCSI-SL003	✓	22 bytes as the ROM
ZuluSCSI	◗	18 bytes; counters always zero
PiSCSI	◗	18 bytes
Snow	◗	18 bytes; counters always zero

✓ matches the v2.0 ROM ◗ differs, usable ✗ not implemented

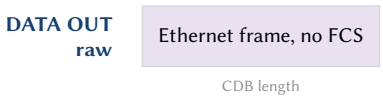
Table 5: READ STATISTICS compatibility

2.5 `0x0A` WRITE

CDB[0]	CDB[1]	CDB[2]	CDB[3]	CDB[4]	CDB[5]
`0A`	0	0	len hi	len lo	format

The format byte selects how the data phase is framed:

Format	Name	Data phase
`0x00`	raw	one Ethernet frame of exactly the CDB length, without FCS (the adapter appends it)
`0x80`	stream	repeated units of a 4-byte prefix (16-bit length, big-endian, then two bytes the receiver ignores) followed by the frame; a prefix with length zero ends the stream



Raw is supported everywhere and is sufficient. The stream format saves the per-command overhead (arbitration, selection, the CDB) when several frames are queued; every implementation surveyed here accepts it.

The 60-byte Ethernet minimum is the driver's responsibility: not every implementation pads, and an undersized frame leaving the adapter is dropped downstream without an error. ARP requests, at 42 bytes, are the common case.

Implementation	Status	Notes
ROM v2.0	✓	reference
ROM v1.3	✓	
BlueSCSI v2	✓	raw and stream
BlueSCSI-SL003	✓	raw and stream
ZuluSCSI	✓	raw and stream
PiSCSI	✓	raw and stream (4-byte stream prefix)
Snow	✓	raw and stream

✓ matches the v2.0 ROM ◗ differs, usable ✗ not implemented

Table 6: WRITE compatibility

2.6 `0x0C` SET MODE / SET ADDRESS

CDB[0]	CDB[1]	CDB[2]	CDB[3]	CDB[4]	CDB[5]
`0C`	0	0	0	mode / len	flavor

DATA OUT station address

6

The flavor byte selects what the command does:

Flavor	Sets	Data phase
`0x80`	mode	none; the mode comes from the CDB
`0x40`	station address	6-byte MAC out
`0xC0`	both	6-byte MAC out

`CDB[4]` carries the mode value for flavor `0x80` and the data length (6) when an address is sent.

The semantics of the mode value are not established by the disassembly; its only observed effect is the runt flag (section 2.3). The original Macintosh driver issues SET MODE during bring-up; drivers verified against the emulators run correctly without it.

A refused address push must be treated as non-fatal: the refusal presents as CHECK CONDITION with sense key 5 (illegal request), and the address read via `0x09` is the one in use. On the BlueSCSI-SL003 fork a combined `0xC0` request is refused entirely, mode-set half included; send the two flavors separately.

Implementation	Status	Notes
ROM v2.0	✓	reference
ROM v1.3	✓	
BlueSCSI v2	✗	accepted and ignored; the address flavor's DATA OUT phase is never entered – a host that sends data anyway faces a phase mismatch
BlueSCSI-SL003	◗	mode flavor stored; address flavor (and the combined form) refused with CHECK CONDITION, radio MAC is fixed
ZuluSCSI	✗	as BlueSCSI v2
PiSCSI	✗	explicit no-op
Snow	✗	rejected: unknown opcodes answer CHECK CONDITION

✓ matches the v2.0 ROM ◗ differs, usable ✗ not implemented

Table 7: SET MODE / SET ADDRESS compatibility

2.7 0x0D ADD MULTICAST

CDB[0]	CDB[1]	CDB[2]	CDB[3]	CDB[4]	CDB[5]
0D	0	0	len hi	len lo	0

DATA OUT	address	address	⋯
	6	6	

One address per six bytes of the length. The ROM walks the whole list and rebuilds its filter; the limit of 16 addresses is the fork's implementation choice and is not confirmed against the ROM. For portability send one command per address: not every implementation walks the list. Whether the ROM preserves the filter across a disable/enable cycle is not established; the fork preserves it. Portable drivers re-send the list after every enable.

Implementation	Status	Notes
ROM v2.0	✓	reference
ROM v1.3	✓	
BlueSCSI v2	◗	installs only the first address per command
BlueSCSI-SL003	✓	whole list, max 16; also programs the radio filter
ZuluSCSI	◗	first address only
PiSCSI	◗	handler present; effect not verified
Snow	◗	one address per command, allocation length ignored

✓ matches the v2.0 ROM ◗ differs, usable ✗ not implemented

Table 8: ADD MULTICAST compatibility

2.8 0x0E ENABLE / DISABLE

CDB[0]	CDB[1]	CDB[2]	CDB[3]	CDB[4]	CDB[5]
0E	0	0	0	0	on/off

No data phase.

`0x80` in the on/off byte enables, `0x00` disables. Enabling flushes the receive queue and resets the Ethernet controller. On real hardware, data commands issued after ENABLE (`0x08`, `0x09`, `0x0A`) answer CHECK CONDITION for up to about 500 ms; retry the first post-enable command until it succeeds. TEST UNIT READY is useless as a settle probe – it answers GOOD throughout. Emulators settle instantly; a driver that waits only as long as an emulator needs will fail on the real device.

Whether the receiver keeps running while disabled is not observable through the SCSI interface: on the BlueSCSI family the disable gate is command-level, but enable flushes the queue either way.

Implementation	Status	Notes
ROM v2.0	✓	reference
ROM v1.3	✓	
BlueSCSI v2	✓	no settle window
BlueSCSI-SL003	✓	no settle window
ZuluSCSI	✓	no settle window
PiSCSI	✓	no settle window
Snow	✓	no settle window

✓ matches the v2.0 ROM ◗ differs, usable ✗ not implemented

Table 9: ENABLE / DISABLE compatibility

2.9 `0x12` INQUIRY

CDB[0]	CDB[1]	CDB[2]	CDB[3]	CDB[4]	CDB[5]
12	0	0	0	alloc	0

DATA IN	standard INQUIRY fields	status
	36	1

Device type 3, vendor `"Dayna  "`, product `"SCSI/Link  "`. The v2.0 ROM clamps the answer at 37 bytes; byte 36 is a status byte: `0x80` enabled, `0x40` mode set. The BlueSCSI-SL003 fork adds two capability bits: `0x01`, bounded READ batches supported, and `0x02`, seamless emission honored (section 2.3). Each is advertised on its own; a driver must not infer one from the other.

Probing guidance: identify the adapter with a *36-byte* request and the vendor and product strings – 36 is the answer every implementation serves in full. Ask for 37 only as a second step, to look for the extension bits: stock BlueSCSI pads the answer to the allocation length with zeros, so byte 36 reads clear there; a short or failed answer likewise means no extension. Treat the undefined bits of byte 36 as reserved: mask exactly the bits being probed.

Implementation	Status	Notes
ROM v2.0	✓	reference
ROM v1.3	✓	37 bytes, byte 36 status
BlueSCSI v2	◗	standard 36 bytes, zero-padded to the allocation length; byte 36 reads zero
BlueSCSI-SL003	✓	37 bytes; byte 36 adds bits `0x01` and `0x02`
ZuluSCSI	◗	as BlueSCSI v2
PiSCSI	◗	exactly 37 bytes; byte 36 always zero
Snow	◗	at most 36 bytes

✓ matches the v2.0 ROM ◗ differs, usable ✗ not implemented

Table 10: INQUIRY compatibility

2.10 Other opcodes

BlueSCSI v2 and ZuluSCSI additionally accept and ignore `0x1A` (mode sense), `0x40` (set MAC) and `0x80` (set mode), and define one vendor opcode for WiFi management (sub-command in `CDB[1]`). No driver should depend on any of these.

3 Driver guidance

3.1 Bring-up sequence

1. **Probe**: INQUIRY, 36 bytes; match vendor and product.
2. **Capability probe** (optional): INQUIRY, 37 bytes; byte 36 bit `0x01` selects bounded reads, bit `0x02` gates the seamless request.
3. **Enable**: `0x0E` with `0x80`.
4. **Settle**: retry the next step on CHECK CONDITION for up to about 500 ms (real hardware only, but always code for it; TEST UNIT READY cannot serve as the probe).
5. **Read the MAC**: `0x09`, 22 bytes, fall back to 18. Read it once and cache it – the command clears the statistics counters. Advertise this MAC. Optionally push it via `0x0C`/`0x40`; treat refusal (sense key 5) as non-fatal.
6. **Poll**: `0x08` on a timer. Use an adaptive cadence: fast while traffic flows or is expected, slow when idle. On shared buses every poll is a full SCSI transaction competing with disk I/O.

3.2 Mode selection

Take bounded mode where the device advertises it: one command per batch, and no special demands on the host. Otherwise blind mode if the SCSI engine can issue several data-in transfers per command, and polled mode if it cannot. Figure 2 shows the decision.

3.3 Polling

What follows is not part of the protocol: it is one way to pace a driver, distilled from lessons learned writing an Open Transport driver for 68k Macs. The constraints it answers are universal, though. The adapter has no interrupt line, so receiving means polling – and every poll is a full SCSI transaction (arbitration, selection, command, status) on a bus most machines share with their boot disks. Poll too slowly and TCP round trips stretch; poll too eagerly and the disks crawl while the machine feels broken.

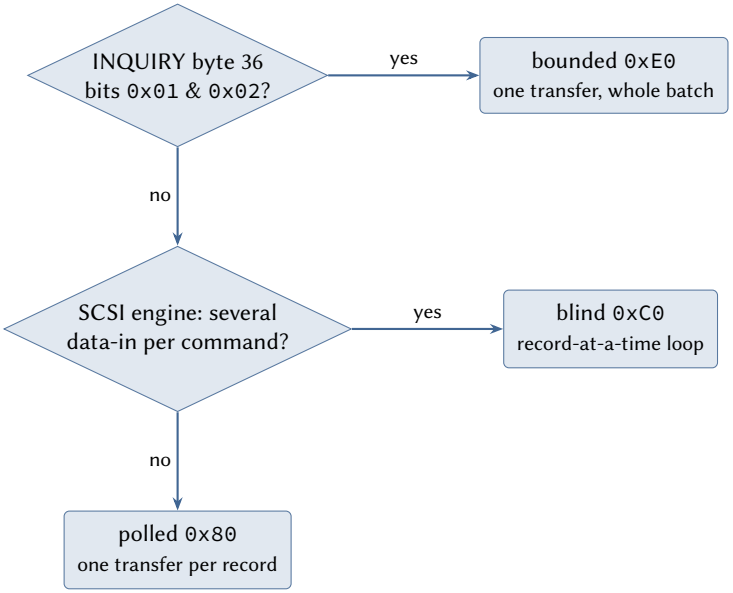


Figure 2: Choosing a receive mode. Polled and bounded need only a plain SCSI API; blind is the special case that needs an engine capable of several data-in transfers per command.

A cadence with three states

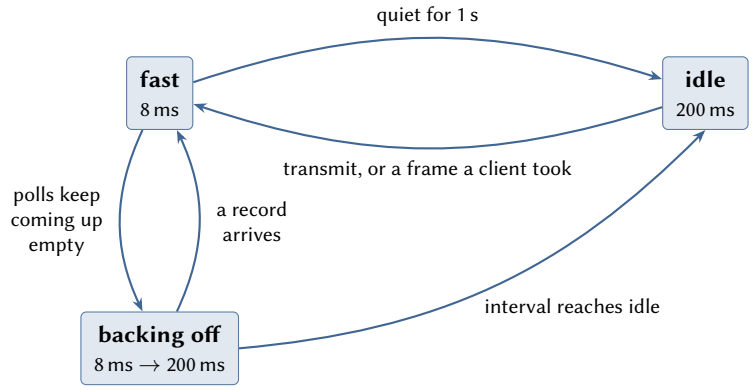


Figure 3: Poll pacing. Traffic a client claimed holds the fast rate; a quiet second returns the idle rate; polls that keep finding nothing back off even while other traffic keeps the link warm.

Two rates bound the behavior: fast while a transfer is in flight, idle otherwise. Two rules make the simple picture work in practice.

First, the fast state must survive longer than a full round trip. During TCP slow start the sender's flights arrive with idle gaps between them; a driver that falls back to the idle rate inside such a gap answers every flight late, the measured round-trip time inflates, and the connection never leaves slow start. A quiet *second* is a safe hysteresis; a quiet few polls is not.

Second, only traffic somebody wanted may hold the fast rate: the driver's own transmits (an answer is probably coming) and received frames a client actually accepted – and of those, only *unicast*. Any healthy LAN delivers broadcast noise (ARP requests for other machines, name-service chatter) about once a second, which is more often than the hysteresis expires: let broadcasts count as traffic and the driver sits at the fast rate forever on an otherwise idle machine.

The third state covers a peer that stalls while keeping the link warm – a closed TCP window, a dead server still answering probes. Traffic holds the fast rate, yet every poll returns empty. Counting consecutive empty polls and stretching the interval exponentially once they exceed a round trip's

worth restores bus courtesy within a few hundred milliseconds, and the first real record snaps the rate back.

Timers poll, completions continue

A transfer completion is the natural moment to start the next piece of work, and for transmits it is the right one: chaining the next queued frame from the previous frame's completion is what keeps a send queue moving at bus speed. Receive polls are the exception. A poll started from a poll's completion produces the next completion, and the loop runs the bus at full speed regardless of need. Give polls their own clock: the timer grants one poll per tick, and a completion – however convenient the moment – never starts one.

Chase evidence, not hope

When a batch ends, the device has said what it knows: the bounded terminator's depth byte (section 2.3), or at least the last more-flag. Read again immediately when several records wait; when the report says one or two, let the next tick take them – an immediate re-read catches the adapter's queue before it has refilled and spends a whole SCSI transaction on a shallow batch. Absent any signal, a batch that filled its budget is the one reliable hint that more is waiting.

Pressure pauses, loss cascades

When the host stack refuses a delivery – flow control, or a failed buffer allocation – the frame is not garbage, it is early. Hold it, stop polling, retry after a tick: the adapter keeps its queue, the transport window closes toward the sender, and the sender slows. Every link in that chain is lossless. Dropping the frame instead converts one moment of local pressure into retransmissions, which arrive as more receive traffic on the same congested path that could not absorb the original.

3.4 SCSI Manager 4.3 hosts: declare the stall points

A blind `SCSIExecIO` promises the SIM that the target streams without pausing. The DaynaPORT breaks that promise at every record: the ROM pauses between a record's header and its payload and again between records. On Quadra-era machines the SIM then fabricates two zero bytes into the buffer at the undeclared stall, at a rate that grows with the pause length – verified with an independent handshake counter on the device side: the wire carried exactly the bytes the device sent while the host delivered two more into memory. Nothing in parity, status or sense reports it; only the record framing exposes it.

The cure is the documented one: `scsiHandshake` declares where the target may stall, and a per-record read is fully described by `{6, 1518, 0}` – header, possible pause, payload. With the pattern declared the same transfers run error-free under the ROM's native pacing. Classic drivers never hit the problem because their chunked TIBs declared the structure implicitly; a 4.3 port that zeroes `scsiHandshake` loses that protection silently.

A bounded batch cannot be declared – record lengths vary, and the pattern is static – so on these hosts bounded reads require the seamless-emission bit (`0x10`), which removes the stalls instead of declaring them. Removing them means all of them: the boundary between two records is a stall as much as the one inside a record, so the whole batch has to leave as a single transfer (section 2.3). The pairing is: bounded plus seamless where the device offers both, per-record polled with the handshake pattern declared against everything else, real ROM included.

3.5 Bus discipline

- Run every transaction to completion (status and message phases), including error paths. An abandoned transaction can wedge the bus.
- When arbitration fails, do not retry in a tight loop at interrupt priority; a receive poll that lost the bus can wait for the next poll tick. Aggressive retry measurably degrades disk throughput on a shared bus.
- Do not invoke completion primitives while the target is still in a data phase; wait for the phase to end first.

A Reference implementation

The complete driver core in portable C, presented block by block.

A.1 Protocol constants

Everything from section 2.3 as numbers. Three have consequences beyond their value.

`DP_READ_ALLOC` is 1524, not 1518: the allocation length has to cover the 6-byte record header *and* a maximum frame. Asking for 1518 clips every full-size frame by six bytes.

`DP_MAX_FRAME` is 1518 and includes the FCS. Every delivery path below subtracts `DP_FCS_LEN`, every length test adds it. A driver that passes the FCS up sees the layer above reject every frame.

`DP_SETTLE_RETRIES` times 10 ms is the 500 ms window real hardware needs after ENABLE. Emulated adapters return immediately, so this constant has no effect until the driver meets real hardware.

```
#define DP_READ_ALLOC   1524            /* header + max frame */
#define DP_MAX_FRAME    1518            /* includes FCS       */
#define DP_HDR_LEN      6
#define DP_ENET_HDR     14              /* dst + src + type   */
#define DP_ENET_MIN     60              /* transmit minimum, no FCS:
                                           64 on the wire */
/* Shortest parseable record: a complete Ethernet header plus the
 * FCS the length includes. Below it there is no header to read and
 * len - FCS underflows. Real hardware sends shorter records (runts,
 * flag 0x80); drop them. */
#define DP_RX_MIN       (DP_ENET_HDR + DP_FCS_LEN)
#define DP_FCS_LEN      4               /* FCS = the Ethernet frame
                                           check sequence, the CRC32
                                           trailing every frame on
                                           the wire. Record lengths
                                           count it; clients never
                                           see it */
#define DP_FLAG_MORE    0x10
#define DP_FLAG_DROPPED 0xFF            /* the four bytes after the
                                           length: real hardware
                                           dropped a packet; recover
                                           via disable/enable */
#define DP_ERR_DROPPED  (-2)            /* receive return code for
                                           that marker */

/*
 * CDB[5] of READ(6) selects the mode. Bit 0x80 is always set; bit
 * 0x40 turns on blind (multi-record) transfers; bit 0x20 is the
 * BlueSCSI-SL003 bounded extension and means nothing to real ROMs.
 * Bit 0x10 (also BlueSCSI-SL003) requests seamless emission: the
 * whole answer in one burst, no pacing gaps and no stall between
```

```
 * records either. Send it from hardware-handshaked hosts only:
 * software-timed blind loops depend on the classic gaps, and
 * Quadra-era SCSI Manager 4.3 machines corrupt on the pauses unless
 * they are declared (see the guide's 4.3 section).
 */
#define DP_MODE_POLLED  0x80
#define DP_MODE_BLIND   0xC0
#define DP_MODE_BOUNDED 0xE0
#define DP_MODE_SEAMLESS  0x10            /* OR into the mode byte */

#define DP_SENSE_LEN        9
#define DP_KEY_ILLEGAL      5
#define DP_STATS_LEN        22          /* MAC + 4 counters: real
                                           SL003 ROM v2.0          */
#define DP_STATS_SHORT      18          /* MAC + 3 counters: the
                                           SCSI emulators (BlueSCSI,
                                           ZuluSCSI, PiSCSI, Snow) */
#define DP_INQ_STD          36          /* identity probe          */
#define DP_INQ_CAP          37          /* capability probe        */
#define DP_INQ_BOUNDED      0x01        /* byte 36: bounded reads */
#define DP_INQ_SEAMLESS     0x02        /* byte 36: seamless honored */
#define DP_SETTLE_RETRIES   50          /* x 10 ms = 500 ms        */
```

A.2 Transport

The platform seam: five SCSI primitives and one delay. Every command except the receive paths runs through `dp_transact`, which enforces the completion rule of section 3.5 in one place. `err` is recorded rather than returned early; an early return here leaves the transaction open on a bus the boot disk shares.

`scsi_complete` returns the SCSI status byte rather than a success flag, so CHECK CONDITION is an ordinary return value. `dp_set_address` below needs that: it must tell a refusal from a transport failure, which is impossible if both collapse to -1.

Polled and bounded receive use `dp_transact` like everything else. Blind receive drives the primitives directly, because it reads several times inside one open command.

```
/* Platform transport, each 0 on success. Once scsi_command has been
 * sent, scsi_complete (status + message phases) must always run, or
 * the bus is left mid-transaction. scsi_complete returns the SCSI
 * status byte (0 = GOOD, 2 = CHECK CONDITION) or negative on error. */
int scsi_select(int target);
int scsi_command(const uint8_t *cdb, int len);
int scsi_data_in(uint8_t *buf, uint32_t len);
int scsi_data_out(const uint8_t *buf, uint32_t len);
int scsi_complete(void);

void platform_delay_ms(int ms);

/* One full transaction: select, command, one optional data phase,
 * complete. dir: 0 none, 1 in, 2 out. Returns the completion status,
 * or -1 on transport failure. */
static int dp_transact(int target, const uint8_t *cdb,
                       uint8_t *buf, uint32_t len, int dir)
{
    int err = 0, status;

    if (scsi_select(target) != 0)
        return -1;
    if (scsi_command(cdb, 6) != 0) {
```

```
        scsi_complete();
        return -1;
    }
    if (dir == 1)
        err = scsi_data_in(buf, len);
    else if (dir == 2)
        err = scsi_data_out(buf, len);

    status = scsi_complete();
    if (err != 0 || status < 0)
        return -1;
    return status;
}
```

A.3 Probing

Two probes. The identity probe asks for exactly 36 bytes, the largest answer every implementation serves in full; the capability probe asks for 37 and reads byte 36. The real ROM answers 37 bytes with the bits clear, stock BlueSCSI zero-pads so the byte reads clear, PiSCSI answers 37 with byte 36 zero, and only the SL003 fork sets them. A clear byte or a short answer is the common case, not an error; the driver uses a different receive mode.

Both probes compare the vendor and product strings including their padding spaces. A prefix comparison would also match devices whose identification starts the same way.

```
/* Identity: 36-byte INQUIRY, match vendor and product. 36 is the
 * answer every implementation serves in full. */
int dp_probe(int target)
{
    static const char vendor[]  = "Dayna   ";          /* 8 chars  */
    static const char product[] = "SCSI/Link       ";  /* 16 chars */
    uint8_t cdb[6] = { 0x12, 0, 0, 0, DP_INQ_STD, 0 };
    uint8_t inq[DP_INQ_STD];
    int i;

    if (dp_transact(target, cdb, inq, DP_INQ_STD, 1) != 0)
        return 0;
    for (i = 0; i < 8; i++)
        if (inq[8 + i] != (uint8_t)vendor[i])
            return 0;
    for (i = 0; i < 16; i++)
        if (inq[16 + i] != (uint8_t)product[i])
            return 0;
    return 1;
}

/* Bounded-batch capability: INQUIRY byte 36 bits 0x01 and 0x02. The
 * real ROM answers 37 bytes with both clear; stock BlueSCSI zero-pads
 * to the allocation length, so the byte reads clear there too. Both
 * bits are required because dp_bounded_read sends the seamless bit:
 * each capability is advertised on its own, and inferring one from
 * the other corrupts on a SCSI Manager 4.3 host if the device honors
 * only the bound. Mask exactly these bits; the others are reserved. */
int dp_probe_bounded(int target)
{
    uint8_t cdb[6] = { 0x12, 0, 0, 0, DP_INQ_CAP, 0 };
    uint8_t inq[DP_INQ_CAP];

    if (dp_transact(target, cdb, inq, DP_INQ_CAP, 1) != 0)
        return 0;
```

```
    return (inq[DP_INQ_STD] & (DP_INQ_BOUNDED | DP_INQ_SEAMLESS))
        == (DP_INQ_BOUNDED | DP_INQ_SEAMLESS) ? 1 : 0;
}
```

A.4 Sense and enable

`dp_request_sense` returns only the key from byte 2. Every implementation reports a key; no two agree on the length or layout of the rest, so a driver that parses further sense data is written against one implementation.

`dp_enable` has two effects not visible in its three lines: the adapter discards its queued frames, and the Ethernet controller resets, opening the settle window. The bring-up block handles both.

```
/* Returns the sense key, or -1. */
int dp_request_sense(int target)
{
    uint8_t cdb[6] = { 0x03, 0, 0, 0, DP_SENSE_LEN, 0 };
    uint8_t sense[DP_SENSE_LEN];

    if (dp_transact(target, cdb, sense, DP_SENSE_LEN, 1) != 0)
        return -1;
    return sense[2] & 0x0F;
}

int dp_enable(int target, int on)
{
    uint8_t cdb[6] = { 0x0E, 0, 0, 0, 0, 0 };

    cdb[5] = on ? 0x80 : 0x00;
    return dp_transact(target, cdb, 0, 0, 0) == 0 ? 0 : -1;
}
```

A.5 MAC address and statistics

One command serves both purposes, and the counters are read-and-clear: every `0x09` zeroes them on the device. A driver that polls this command to monitor the link destroys the deltas it is reading, and a second consumer of the statistics sees zeros. The bring-up block therefore reads it once for the MAC and caches the result; later calls belong only where the caller consumes the counters.

The length fallback covers the population split: real v2.0 hardware and the SL003 fork answer 22 bytes (MAC plus four counters), the other emulators 18 (MAC plus three). Asking for 22 and retrying with 18 costs one extra transaction at bring-up and keeps the fourth counter where it exists. The MAC occupies the first six bytes in both cases.

```
/*
 * MAC and statistics share command 0x09, and the counters are
 * read-and-clear: call this once at bring-up for the MAC, and from
 * then on only when consuming the counter deltas.
 *
 * Fills mac[6]; stats may be 0 and must hold 16 bytes otherwise --
 * the ROM answers 22 even to a driver developed against emulators.
 * Returns bytes of statistics data (16 or 12) or -1. The buffer is
 * zeroed first: a transport that accepts short transfers answers
 * the 22-byte request with an emulator's 18, and the tail must not
 * be stale.
 */
int dp_read_stats(int target, uint8_t *mac, uint8_t *stats)
{
```

```
    uint8_t cdb[6] = { 0x09, 0, 0, 0, DP_STATS_LEN, 0 };
    uint8_t buf[DP_STATS_LEN];
    int i, len = DP_STATS_LEN;

    for (i = 0; i < DP_STATS_LEN; i++)
        buf[i] = 0;
    if (dp_transact(target, cdb, buf, DP_STATS_LEN, 1) != 0) {
        cdb[4] = DP_STATS_SHORT;        /* emulated adapters: 18 */
        len = DP_STATS_SHORT;
        if (dp_transact(target, cdb, buf, DP_STATS_SHORT, 1) != 0)
            return -1;
    }
    for (i = 0; i < 6; i++)
        mac[i] = buf[i];
    if (stats)
        for (i = 6; i < len; i++)
            stats[i - 6] = buf[i];
    return len - 6;
}
```

A.6 Station address and multicast

`dp_set_address` returns three values because pushing the station address is optional and frequently refused: emulated adapters bridge to a fixed MAC and answer CHECK CONDITION with sense key 5. The address already read from `0x09` remains correct. A two-valued return leaves the caller choosing between aborting bring-up over a refusal and ignoring transport faults.

The function sends the address flavor alone. The SL003 fork refuses the combined `0xC0` form entirely, mode-set half included, so the two flavors go as separate commands.

`dp_add_multicast` sends one address per command although the ROM accepts a list: BlueSCSI and ZuluSCSI install the first address and drop the rest without an error, which surfaces later as missing multicast traffic. The caller re-sends the whole set after every enable, since filter persistence across a disable/enable cycle is unestablished for the ROM and differs between implementations.

```
/* Push the station address. A refusal (CHECK CONDITION, sense key 5)
 * is normal on emulated adapters (their MAC is fixed) and not an
 * error: the address from
 * dp_read_stats is the one in use. Send the mode and address flavors
 * separately; a combined 0xC0 is refused whole on the SL003 fork.
 * Returns 0 sent, 1 refused, -1 transport error. */
int dp_set_address(int target, const uint8_t *mac)
{
    uint8_t cdb[6] = { 0x0C, 0, 0, 0, 6, 0x40 };
    uint8_t addr[6];
    int i, status;

    for (i = 0; i < 6; i++)
        addr[i] = mac[i];
    status = dp_transact(target, cdb, addr, 6, 2);
    if (status < 0)
        return -1;
    if (status != 0)
        return dp_request_sense(target) == DP_KEY_ILLEGAL ? 1 : -1;
    return 0;
}

/* One address per command: not every implementation walks a list.
 * Re-send the whole set after every enable; whether the filter
 * survives a disable/enable cycle is implementation-defined. */
```

```
int dp_add_multicast(int target, const uint8_t *addr)
{
    uint8_t cdb[6] = { 0x0D, 0, 0, 0, 6, 0 };
    uint8_t a[6];
    int i;

    for (i = 0; i < 6; i++)
        a[i] = addr[i];
    return dp_transact(target, cdb, a, 6, 2) == 0 ? 0 : -1;
}
```

A.7 Transmit

Two paths: raw mode, one frame per command, and the stream format, several frames per command at the cost of assembling the batch – one prefix and one padded frame at a time, closed by the zero-length terminator – in a buffer first. Both are single data-out transfers, so both run on a plain SCSI API.

The padding loops enforce the 60-byte Ethernet minimum of section 2.5. The FCS is not sent: the adapter appends it, so the lengths are the frame *without* its checksum – the opposite convention from the receive path.

```
/* One raw frame, no FCS (the adapter appends it). Pads to the
 * Ethernet minimum; not every implementation pads for you. frame
 * must have room for DP_ENET_MIN bytes. */
int dp_send(int target, uint8_t *frame, uint32_t len)
{
    uint8_t cdb[6] = { 0x0A, 0, 0, 0, 0, 0 };

    if (len < DP_ENET_HDR || len > DP_MAX_FRAME - DP_FCS_LEN)
        return -1;
    while (len < DP_ENET_MIN)
        frame[len++] = 0;
    cdb[3] = (uint8_t)(len >> 8);
    cdb[4] = (uint8_t)len;
    return dp_transact(target, cdb, frame, len, 2) == 0 ? 0 : -1;
}

/*
 * Several frames in one command: stream format 0x80. Each frame is
 * led by a 4-byte prefix (16-bit big-endian length, two bytes the
 * device ignores); a zero-length prefix ends the stream. The device
 * takes its lengths from the prefixes and ignores the CDB length in
 * this format, so it stays zero. One data-out phase, so a plain SCSI
 * API carries it.
 *
 * next feeds the batch from the driver's queue: it sets *frame and
 * returns the length, 0 when the queue is empty. Frames are taken
 * only while a maximum-size frame still fits buf, so nothing is
 * fetched that cannot be sent; call again until the queue drains.
 * Padding to the Ethernet minimum happens in buf, as in dp_send.
 * buf must hold one maximum frame with its prefix and the
 * terminator: 1522 bytes.
 *
 * Returns the number of frames sent, or -1.
 */
int dp_send_batch(int target, uint8_t *buf, uint32_t cap,
                  uint32_t (*next)(const uint8_t **frame))
{
    uint8_t cdb[6] = { 0x0A, 0, 0, 0, 0, 0x80 };
    const uint8_t *frame;
    uint32_t off = 0, len, padded, j;
```

```
    int count = 0;

    if (cap < 4 + (DP_MAX_FRAME - DP_FCS_LEN) + 4)
        return -1;

    while (off + 4 + (DP_MAX_FRAME - DP_FCS_LEN) + 4 <= cap) {
        len = next(&frame);
        if (len == 0)
            break;
        if (len < DP_ENET_HDR || len > DP_MAX_FRAME - DP_FCS_LEN)
            return -1;
        padded = len < DP_ENET_MIN ? DP_ENET_MIN : len;
        buf[off++] = (uint8_t)(padded >> 8);
        buf[off++] = (uint8_t)padded;
        buf[off++] = 0;
        buf[off++] = 0;
        for (j = 0; j < len; j++)
            buf[off + j] = frame[j];
        for (; j < padded; j++)
            buf[off + j] = 0;
        off += padded;
        count++;
    }
    if (count == 0)
        return 0;

    buf[off++] = 0;                     /* zero-length terminator */
    buf[off++] = 0;
    buf[off++] = 0;
    buf[off++] = 0;

    return dp_transact(target, cdb, buf, off, 2) == 0 ? count : -1;
}
```

A.8 Receive, polled

One record per command through `dp_transact`: one data-in of `DP_READ_ALLOC` bytes, header parsed from the front of the buffer, frame delivered from an offset without a copy. The more-flag decides only whether to issue the next command, and the burst wrapper caps how many one poll may spend.

No length is validated before a transfer, because the transfer size was the host's own choice; the check that remains stops the caller reading past what arrived.

```
/*
 * Polled receive: one record per command.
 *
 * Ask for a whole record in one transfer and parse the header out of
 * the front of the buffer. The device sends 6 + len bytes and ends
 * the data phase, so the transport reports a short transfer; a plain
 * SCSI API returns success with a residual.
 *
 * buf must hold DP_READ_ALLOC bytes. Returns frame length (0 = queue
 * empty), -1 on error, DP_ERR_DROPPED on the dropped-packet marker
 * (recover with a disable/enable cycle). *more: issue the next read
 * now.
 */
int dp_poll_one(int target, uint8_t *buf, int *more)
{
    uint8_t cdb[6] = { 0x08, 0, 0,
                       DP_READ_ALLOC >> 8, DP_READ_ALLOC & 0xFF,
```

```
                          DP_MODE_POLLED };
    uint32_t len;

    *more = 0;
    if (dp_transact(target, cdb, buf, DP_READ_ALLOC, 1) != 0)
        return -1;

    if (buf[2] == DP_FLAG_DROPPED && buf[3] == DP_FLAG_DROPPED
     && buf[4] == DP_FLAG_DROPPED && buf[5] == DP_FLAG_DROPPED)
        return DP_ERR_DROPPED;

    /* No validation is possible before the transfer here, and none
     * is needed: the transfer was bounded by the buffer size we
     * asked for. A nonsense length is still refused, because the
     * caller must not read past what arrived. */
    len = ((uint32_t)buf[0] << 8) | buf[1];
    if (len > DP_MAX_FRAME)
        return -1;

    *more = (buf[5] & DP_FLAG_MORE) ? 1 : 0;
    return (int)len;
}

/* The frame starts after the header; no copy is needed. */
int dp_poll_burst(int target, int max_frames,
                  void (*deliver)(const uint8_t *frame, int len))
{
    uint8_t buf[DP_READ_ALLOC];
    int n, more, count = 0;

    if (max_frames <= 0)
        return 0;

    do {
        n = dp_poll_one(target, buf, &more);
        if (n < 0)
            return n;
        if (n >= DP_RX_MIN)
            deliver(buf + DP_HDR_LEN, n - DP_FCS_LEN);
        count++;
    } while (more && count < max_frames);

    return count;
}
```

A.9 Receive, bounded

A whole batch in one command and one transfer. The budget goes out in the allocation length, the device stops before exceeding it, and the records are walked in memory afterwards. Every bound in that walk is against the budget the host chose, never against a length the device supplied: a record claiming to run past the end of the buffer stops the walk rather than steering it.

Run this only after `dp_probe_bounded` returned true. A device without the extension does not reject `0xE0`: bit `0x20` is ignored, the read behaves as plain blind, and the batch runs past the budget the buffer was sized to.

```c
/*
 * Bounded receive: one command, one transfer, a whole batch.
 *
 * The allocation length bounds the batch, so the device never sends
 * more than the buffer holds. Take the batch in one transfer and
 * walk the records in memory afterwards. Only available where the
 * INQUIRY capability bit advertises it.
 *
 * The batch ends exactly as a blind batch ends. Drained queue: the
 * last record’s more-flag is clear, nothing follows. Budget or
 * record cap: records keep truthful flags and a zero-length
 * terminator closes the batch, its byte 2 reporting the queue depth
 * (records, clamped to 255) -- read again immediately when several
 * wait, let one or two deepen for the next poll. Older firmware
 * sends zeros there, which reads as "no signal".
 *
 * Returns the number of frames delivered, or -1. *pending: queue
 * depth from the terminator, 0 when the batch drained the queue or
 * the device gave no signal.
 */
int dp_bounded_read(int target, uint8_t *buf, uint32_t budget,
                    void (*deliver)(const uint8_t *frame, int len),
                    int *pending)
{
    /* Seamless emission assumed wanted here: a host using bounded
     * mode has a hardware-handshaked engine (that is why it can take
     * the batch in one transfer). Drop the bit if yours is not. */
    uint8_t cdb[6] = { 0x08, 0, 0, 0, 0,
                       DP_MODE_BOUNDED | DP_MODE_SEAMLESS };
    uint32_t off = 0, len;
    int count = 0;

    *pending = 0;

    if (budget > 0xFFFF)
        budget = 0xFFFF;
    cdb[3] = (uint8_t)(budget >> 8);
    cdb[4] = (uint8_t)budget;

    if (dp_transact(target, cdb, buf, budget, 1) != 0)
        return -1;

    /* Walk the records the device packed into the buffer. Every
     * bound here is against the budget, never against a length the
     * device supplied. */
    while (off + DP_HDR_LEN <= budget) {
        len = ((uint32_t)buf[off] << 8) | buf[off + 1];
        if (len == 0) {                 /* closing terminator */
            *pending = buf[off + 2];    /* queue depth, 0 = none  */
            if ((buf[off + 5] & DP_FLAG_MORE) && *pending == 0)
                *pending = 1;           /* flag without depth byte */
            break;
        }
        if (len > DP_MAX_FRAME || off + DP_HDR_LEN + len > budget)
            return count > 0 ? count : -1;

        if (len >= DP_RX_MIN) {
            deliver(buf + off + DP_HDR_LEN, (int)(len - DP_FCS_LEN));
            count++;
        }
        if (!(buf[off + 5] & DP_FLAG_MORE))
            break;
        off += DP_HDR_LEN + len;
```

```
    }
    return count;
}
```

A.10 Receive, blind

The special case: the only path here that needs a SCSI engine able to issue several data-in transfers per command (section 2.3). The loop implements the contract of section 2.3, and has four exits.

A record whose more-flag is clear ends the batch. A zero-length record ends it as well: the device cut the batch short at its record cap. A failed header read is not an error – the data phase ended after a complete record despite the flag, as PiSCSI does – so the loop records success and falls through to completion. Only a length beyond 1518 is a protocol error.

The loop never stops because the host is out of room. `has_room` lets the consumer decline a frame, but the record is read off the bus first and dropped afterwards. Ending the transaction with the target still in DATA IN risks the bus; a dropped frame costs a retransmission. One scratch buffer serves as both delivery buffer and discard sink, so batch size does not depend on host memory.

```
/*
 * Blind mode: records back to back in one command.
 *
 * SPECIAL CASE. Every other path here uses a plain SCSI API -- one
 * command, one data-in of at most N bytes, short transfers fine.
 * Blind mode cannot: nothing bounds the batch in advance, so the
 * host must read each record's header, learn its length, and read
 * the payload, all inside one open command. A SCSI engine that
 * cannot issue several data-in transfers per command has no way to
 * run this mode; use polled, or bounded where it is advertised.
 *
 * The host must not end the batch; the device does, via a clear
 * more-flag or a zero-length terminator record. If the data phase
 * ends after a complete record despite a set more-flag (PiSCSI),
 * that is end of batch, not an error. has_room lets the consumer
 * refuse a frame without ending the batch; the record is read
 * either way. Returns frames delivered, -1 on error, or
 * DP_ERR_DROPPED on the dropped-packet marker.
 */
int dp_blind_burst(int target,
                   int (*has_room)(uint32_t len),
                   void (*deliver)(const uint8_t *frame, int len))
{
    uint8_t cdb[6] = { 0x08, 0, 0,
                       DP_READ_ALLOC >> 8, DP_READ_ALLOC & 0xFF,
                       DP_MODE_BLIND };
    uint8_t hdr[DP_HDR_LEN];
    uint8_t frame[DP_MAX_FRAME];        /* also the discard sink */
    uint32_t len;
    int err, status, count = 0;

    if (scsi_select(target) != 0)
        return -1;
    if (scsi_command(cdb, 6) != 0) {
        scsi_complete();
        return -1;
    }

    for (;;) {
        err = scsi_data_in(hdr, DP_HDR_LEN);
        if (err != 0) {
```

```
            err = 0;                    /* phase ended: batch over */
            break;
        }

        if (hdr[2] == DP_FLAG_DROPPED && hdr[3] == DP_FLAG_DROPPED
         && hdr[4] == DP_FLAG_DROPPED && hdr[5] == DP_FLAG_DROPPED) {
            err = DP_ERR_DROPPED;
            break;
        }

        len = ((uint32_t)hdr[0] << 8) | hdr[1];
        if (len > DP_MAX_FRAME) {
            err = -1;
            break;
        }
        if (len == 0)
            break;                      /* empty queue / terminator */

        err = scsi_data_in(frame, len);
        if (err != 0)
            break;

        if (len >= DP_RX_MIN && has_room(len)) {
            deliver(frame, (int)(len - DP_FCS_LEN));
            count++;
        }

        if (!(hdr[5] & DP_FLAG_MORE))
            break;
    }

    status = scsi_complete();
    if (err == DP_ERR_DROPPED)
        return DP_ERR_DROPPED;
    if (status != 0 || err != 0)
        return count > 0 ? count : -1;
    return count;
}
```

A.11 Bring-up and recovery

The sequence of section 3.1 in code. The retry loop repeats `dp_read_stats`, not TEST UNIT READY: during the settle window the adapter answers CHECK CONDITION to data commands while TEST UNIT READY keeps answering GOOD, so a driver probing with TUR issues its first read into a controller that is still resetting. The statistics read exits the loop when the device can serve data, and yields the MAC from the same command.

`dp_recover` is the error policy. Sense key 5 after a data command means the interface reset underneath the driver: re-run bring-up. Anything else is transient and the next poll handles it. The receive paths return `DP_ERR_DROPPED` for the all-ones dropped-packet marker of section 2.3; the recovery is the same disable/enable cycle, without waiting for a sense key.

```c
/*
 * Probe, enable, wait out the settle window, learn the MAC.
 * After ENABLE, real hardware answers CHECK CONDITION on data
 * commands for up to ~500 ms; the first 0x09 doubles as the settle
 * probe (TEST UNIT READY answers GOOD throughout and probes
 * nothing). Emulated adapters settle instantly, so the loop exits
 * on its first pass there.
 *
 * Fills mac[6] and *bounded. Returns 0, or -1 when no adapter.
 */
int dp_bringup(int target, uint8_t *mac, int *bounded)
{
    int i;

    if (!dp_probe(target))
        return -1;
    *bounded = dp_probe_bounded(target);

    if (dp_enable(target, 1) != 0)
        return -1;

    for (i = 0; i < DP_SETTLE_RETRIES; i++) {
        if (dp_read_stats(target, mac, 0) >= 0)
            return 0;
        platform_delay_ms(10);
    }
    return -1;
}

/*
 * After CHECK CONDITION on a data command: sense, and re-run enable
 * and settle when the interface reset underneath us. Also the
 * recovery for the all-ones dropped-packet flag.
 */
int dp_recover(int target, uint8_t *mac)
{
    int key = dp_request_sense(target);
    int bounded;

    if (key < 0)
        return -1;
    if (key == DP_KEY_ILLEGAL)
        return dp_bringup(target, mac, &bounded);
    return 0;                           /* transient: next poll */
}
```

B References

- BlueSCSI-SL003 firmware fork (SL003 reimplementation from the v2.0 ROM disassembly), release downloads: https://github.com/ingpaschke/BlueSCSI-v2/releases/tag/daynaport-sl003-v3.0
- BlueSCSI v2: https://github.com/BlueSCSI/BlueSCSI-v2
- ZuluSCSI: https://github.com/ZuluSCSI/ZuluSCSI-firmware
- PiSCSI: https://github.com/PiSCSI/piscsi
- Snow: https://snowemu.com